

For your situation (7 years experience, infra background, targeting Google-level roles), you should approach DSA very differently from freshers.

You do NOT need:

- Codeforces grandmaster grinding
- extreme competitive programming
- 1000 random problems

You need:

- ✓ interview-oriented problem solving
 - ✓ clean Python implementation
 - ✓ pattern recognition
 - ✓ communication during coding
-

RIGHT WAY TO LEARN DSA WITH PYTHON

Goal

Become able to:

- solve medium-level interview problems
- explain thought process clearly
- write bug-free Python
- handle coding rounds confidently

That's enough for many platform/SRE/cloud interviews.

BEST APPROACH FOR YOU

Step 1 — Learn Python for DSA First

Before LeetCode.

You should become comfortable with:

Python Features Used in Interviews

MUST KNOW

lists

```
arr = [1,2,3]
```

dict/hashmap

```
mp = {}
```

set

```
s = set()
```

tuple

```
t = (1,2)
```

deque

```
from collections import deque
```

heap

```
import heapq
```

sorting

```
arr.sort()
```

lambda

```
sorted(arr, key=lambda x: x[1])
```

VERY IMPORTANT

Learn:

- list comprehension
- enumerate
- zip
- collections module
- defaultdict
- Counter
- heapq

These save huge time in interviews.

Step 2 — Learn DSA BY PATTERNS

This is the MOST IMPORTANT advice.

Do NOT do random questions.

Instead:

- learn one pattern
 - solve 5–10 questions on same pattern
-

BEST ORDER FOR YOU

PHASE 1 — EASY CONFIDENCE BUILDING

1. Arrays

Learn

- traversal
- prefix sum
- two pointers

Questions

- Two Sum
 - Best Time to Buy Stock
 - Move Zeroes
-

2. Strings

Learn

- hashmap counting
- sliding window

Questions

- Valid Anagram
 - Longest Substring Without Repeating Characters
-

3. Hashmaps + Sets

VERY IMPORTANT.

Most infra interviews use these heavily.

PHASE 2 — CORE INTERVIEW PATTERNS

4. Sliding Window

EXTREMELY IMPORTANT.

Questions

- Longest substring
 - Max sum subarray
 - Permutation in string
-

5. Stack + Queue

Questions

- Valid Parentheses
 - Daily Temperatures
 - Min Stack
-

6. Binary Search

Questions

- Search rotated array
 - First bad version
-

7. Linked Lists

Mostly:

- reverse
 - cycle detection
-

PHASE 3 — IMPORTANT FOR GOOGLE-LEVEL THINKING

8. Trees

VERY important.

Learn

- DFS
- BFS
- recursion

Questions

- Level order traversal
 - Lowest common ancestor
 - Diameter of binary tree
-

9. Heaps

Very common in infra/system questions.

Questions

- Top K elements
 - Merge K sorted lists
-

10. Graphs

Important for:

- distributed systems thinking
- dependency problems

Learn

- BFS
 - DFS
 - topological sort
-



BEST LEARNING RESOURCE FOR YOU



NeetCode (BEST)

NeetCode Roadmap

This is PERFECT for your situation.

Why?

- structured
 - pattern-based
 - not overwhelming
 - Python-friendly
-

BEST STRATEGY FOR EACH QUESTION

STEP 1 — Understand Problem

Ask:

- input?
 - output?
 - constraints?
 - brute force?
-

STEP 2 — Speak Your Thinking

Practice this while recording.

Example:

“Brute force is $O(n^2)$.

We can optimize using hashmap because lookup is $O(1)$.”

This matters hugely in interviews.

STEP 3 — Write Clean Python

Focus on:

- readability
- correctness
- edge cases

NOT fancy tricks.

STEP 4 — Analyze Complexity

Always explain:

Time Complexity: $O(n)$

Space Complexity: $O(n)$

STEP 5 — Test Manually

Walk through:

- empty input
 - duplicates
 - edge cases
-



IMPORTANT FOR SENIOR ENGINEERS

You are NOT expected to:

- memorize 500 problems
- compete with fresh grads in competitive programming

Interviewers expect:

- ✓ structured thinking
- ✓ debugging ability

- ✓ clarity
 - ✓ correctness
 - ✓ communication
-

YOUR IDEAL DAILY PLAN

Since you work night shift:

Time	Task
45 mins	DSA learning
45 mins	Solve 1–2 problems
15 mins	Explain solution verbally

MOST IMPORTANT PYTHON TOPICS FOR DSA

Must Master

dict
set
list
deque
heapq
Counter
defaultdict
sorting

lambda



HOW TO RECORD YOURSELF

VERY effective method:

Example Recording

Question

“Longest substring without repeating characters”

Explain:

1. brute force
2. optimization
3. sliding window logic
4. complexity
5. edge cases

This simulates real interviews EXTREMELY well.



REALISTIC TARGET FOR YOU

You do NOT need:

✗ 500 LeetCode problems

You probably need:

✓ 120–180 quality problems deeply understood

That is enough for many senior infra/platform interviews when combined with:

- system design
- cloud depth
- Kubernetes expertise
- communication
- production experience



MOST IMPORTANT ADVICE

Don't separate:

- DSA
- system design
- infra knowledge

Combine them.

Example:

- Graphs → dependency resolution
- Heaps → scheduling systems
- Queues → distributed workers
- Sliding window → metrics/log processing

This creates senior-level intuition instead of “exam preparation.”